



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение  
высшего образования

«МИРЭА – Российский технологический университет»

**РТУ МИРЭА**

---

## **ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ**

по дисциплине «Моделирование сред и разработка приложений виртуальной  
и дополненной реальности»

### **Практическая работа № 2**

Студент группы *ИКБО-50-23, Павлов Н.С.*

\_\_\_\_\_  
(подпись)

Старший  
преподаватель *Горбатов Г.В.*

\_\_\_\_\_  
(подпись)

Отчет представлен «\_\_» \_\_\_\_\_ 2026 г.

Москва 2026 г.

1. В рамках выбранной темы проекта был спроектирован (частично) тестовый уровень, который в последующем будет являться обучающим. Структура объектов и макет уровня представлены на рис. 1.

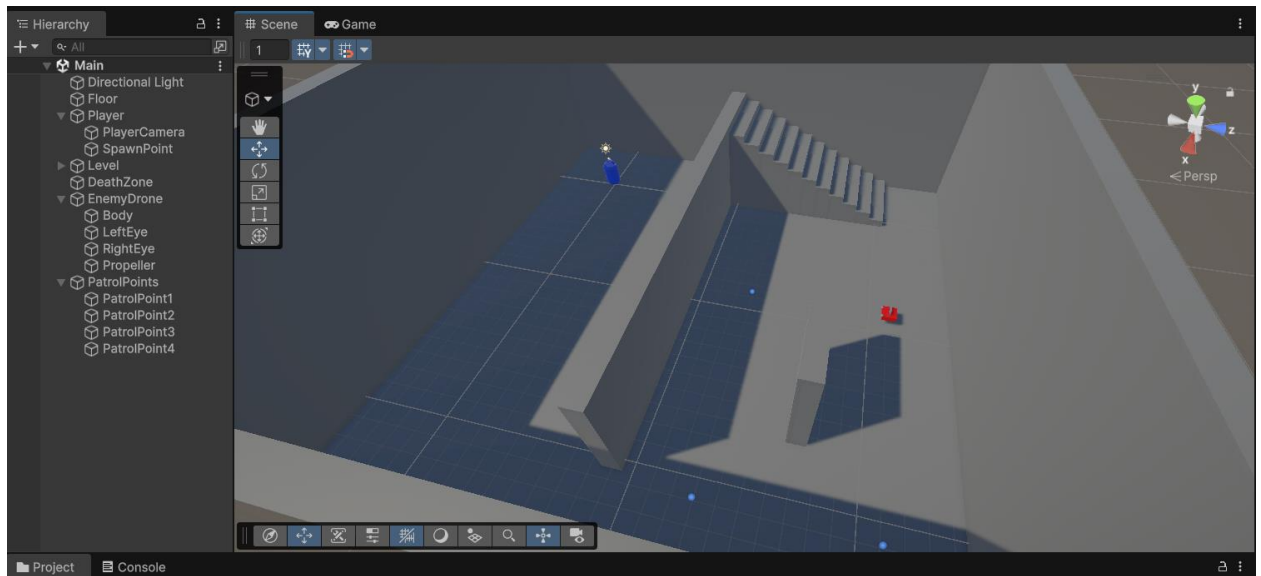


Рисунок 1 — Иерархия объектов и сцена

2. Роль персонажа выполняет пустой объект с примененным Rigidbody и Capsule Collider (для тестирования и наглядности также отображен меш капсулы). Дочерними объектами к Персонажу выступают камера и точка появления объектов, что способствует групповому перемещению и изменению параметров объектов. Структура объектов и макет уровня представлены на рис. 2.



Рисунок 2 — Свойства объекта Player

3. Для управления персонажем был реализован кастомный контроллер (листинг 1). PlayerController помимо передвижения содержит в себе

управление другими переменными персонажа (здоровье).

*Листинг 1 - Код скрипта PlayerController.cs на C#*

```
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    [Header("Настройки передвижения")]
    public float walkSpeed = 5f;
    public float runSpeed = 8f;
    public float jumpForce = 5f;
    public float mouseSensitivity = 2f;

    [Header("Настройки здоровья")]
    public float maxHealth = 100f;
    private float currentHealth;

    [Header("Компоненты")]
    public Transform playerCamera;

    private Rigidbody rb;
    private float rotationX = 0f;
    private bool isGrounded;
    private Vector3 moveDirection;
    private float currentSpeed;

    private void Awake()
    {
        rb = GetComponent<Rigidbody>();

        if (playerCamera == null)
        {
            playerCamera = GetComponentInChildren<Camera>().transform;
        }

        rb.freezeRotation = true;

        currentHealth = maxHealth;

        Cursor.lockState = CursorLockMode.Locked;
        Cursor.visible = false;
    }

    private void Update()
    {
        HandleInput();
        HandleMouseLook();
    }

    private void FixedUpdate()
    {
        CheckGround();
        HandleMovement();
    }
}
```

```

private void HandleInput()
{
    float horizontal = Input.GetAxis("Horizontal");
    float vertical = Input.GetAxis("Vertical");

    Vector3 forward = transform.forward;
    Vector3 right = transform.right;

    forward.y = 0f;
    right.y = 0f;

    forward.Normalize();
    right.Normalize();

    moveDirection = (forward * vertical + right * horizontal).normalized;

    if (Input.GetKey(KeyCode.LeftShift))
    {
        currentSpeed = runSpeed;
    }
    else
    {
        currentSpeed = walkSpeed;
    }

    if (Input.GetButtonDown("Jump") && isGrounded)
    {
        Jump();
    }

    if (Input.GetKeyDown(KeyCode.Escape))
    {
        Cursor.lockState = CursorLockMode.None;
        Cursor.visible = true;
    }
}

private void HandleMouseLook()
{
    float mouseX = Input.GetAxis("Mouse X") * mouseSensitivity;
    float mouseY = Input.GetAxis("Mouse Y") * mouseSensitivity;

    transform.Rotate(Vector3.up * mouseX);

    rotationX -= mouseY;
    rotationX = Mathf.Clamp(rotationX, -90f, 90f);

    playerCamera.localRotation = Quaternion.Euler(rotationX, 0f, 0f);
}

private void HandleMovement()
{
    Vector3 targetVelocity = moveDirection * currentSpeed;

    targetVelocity.y = rb.linearVelocity.y;

```

```

        rb.linearVelocity = targetVelocity;
    }

    private void CheckGround()
    {
        Vector3 rayOrigin = transform.position + Vector3.up * 0.1f;
        float rayLength = 1.1f;

        Debug.DrawRay(rayOrigin, Vector3.down * rayLength, Color.green);

        isGrounded = Physics.Raycast(rayOrigin, Vector3.down, rayLength);
    }

    private void Jump()
    {
        rb.linearVelocity = new Vector3(rb.linearVelocity.x, 0f,
rb.linearVelocity.z);

        rb.AddForce(Vector3.up * jumpForce, ForceMode.Impulse);

        Debug.Log("Прыжок!");
    }

    public void TakeDamage(float damage)
    {
        currentHealth -= damage;
        Debug.Log($"Получен урон! Здоровье: {currentHealth}");

        if (currentHealth <= 0)
        {
            Die();
        }
    }

    private void Die()
    {
        Debug.Log("Игрок погиб!");

        this.enabled = false;

        rb.constraints = RigidbodyConstraints.FreezeAll;

        Invoke("Respawn", 2f);
    }

    private void Respawn()
    {
        transform.position = new Vector3(0, 1, 0);

        currentHealth = maxHealth;

        this.enabled = true;
    }

```

```

        rb.constraints = RigidbodyConstraints.FreezeRotation;

        rb.freezeRotation = true;

        Debug.Log("Игрок возродился!");
    }

    private void OnTriggerEnter(Collider other)
    {
        if (other.gameObject.name == "DeathZone")
        {
            TakeDamage(maxHealth);
        }
    }
}

```

4. В качестве противника реализован дрон, который «патрулирует область» по заданному массиву точек. При приближении игрока, дрон начинает преследование и атаку персонажа, тем самым уменьшая здоровье и убивая его.

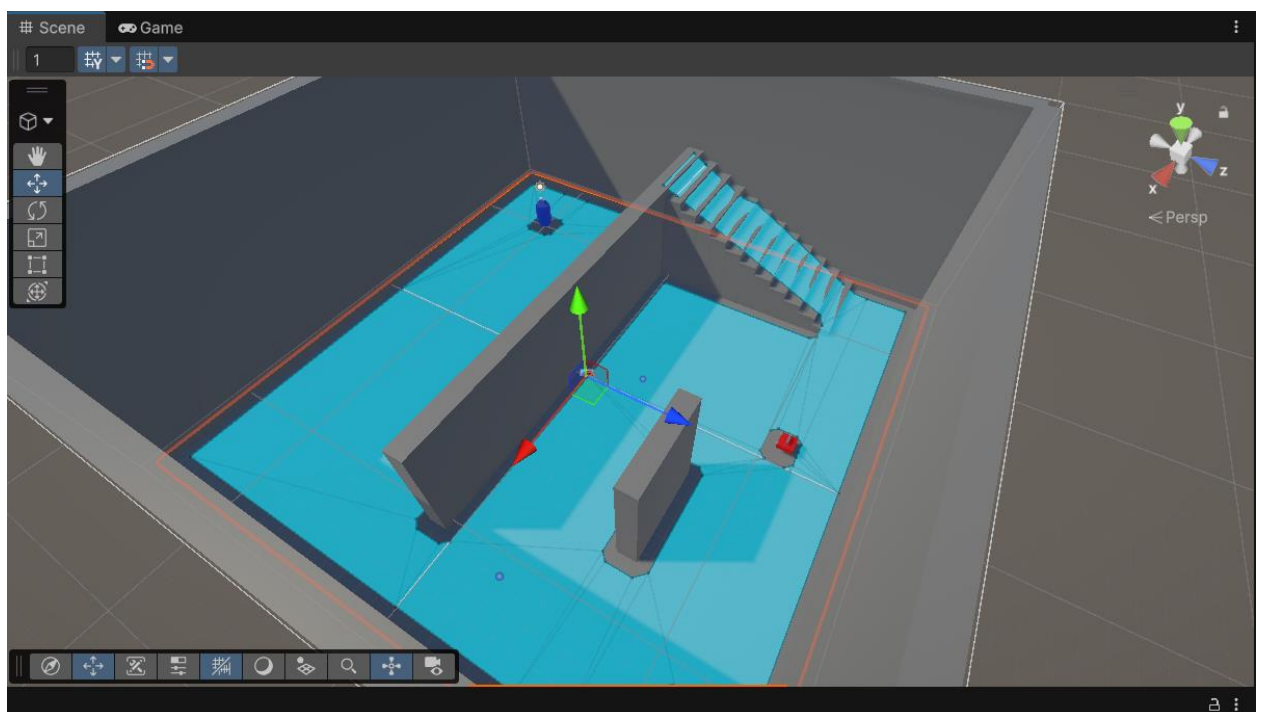


Рисунок 3 — Область навигации NPC

5. Код реализации противника продемонстрирован в листинге 2.

*Листинг 2 - Код скрипта EnemyDrone.cs на C#*

```

using UnityEngine;
using UnityEngine.AI;

```

```

public class EnemyDrone : MonoBehaviour
{
    [Header("Настройки патрулирования")]
    public Transform[] patrolPoints;
    public float waitTime = 2f;

    [Header("Настройки преследования")]
    public float detectionRange = 10f;
    public float attackRange = 2f;
    public float attackDamage = 10f;

    private NavMeshAgent agent;
    private Transform player;
    private PlayerController playerController;
    private int currentPointIndex = 0;
    private float waitTimer = 0f;
    private bool isWaiting = false;

    private enum State
    {
        Patrol,
        Chase,
        Attack
    }

    private State currentState = State.Patrol;

    private void Start()
    {
        agent = GetComponent<NavMeshAgent>();

        GameObject playerObj = GameObject.FindGameObjectWithTag("Player");
        if (playerObj != null)
        {
            player = playerObj.transform;
            playerController = playerObj.GetComponent<PlayerController>();
        }

        if (patrolPoints.Length > 0)
        {
            agent.SetDestination(patrolPoints[0].position);
        }
    }

    private void Update()
    {
        if (player == null || playerController == null) return;

        float distanceToPlayer = Vector3.Distance(transform.position, player.position);

        switch (currentState)
        {
            case State.Patrol:
                Patrol(distanceToPlayer);

```



```

        break;
    case State.Chase:
        Chase(distanceToPlayer);
        break;
    case State.Attack:
        Attack(distanceToPlayer);
        break;
    }
}

private void Patrol(float distanceToPlayer)
{
    if (distanceToPlayer < detectionRange)
    {
        currentState = State.Chase;
        return;
    }

    if (patrolPoints.Length == 0) return;

    if (isWaiting)
    {
        waitTimer += Time.deltaTime;
        if (waitTimer >= waitTime)
        {
            isWaiting = false;
            GoToNextPoint();
        }
        return;
    }

    if (!agent.pathPending && agent.remainingDistance < 0.5f)
    {
        isWaiting = true;
        waitTimer = 0f;
    }
}

private void Chase(float distanceToPlayer)
{
    if (distanceToPlayer > detectionRange * 1.5f)
    {
        currentState = State.Patrol;

        if (patrolPoints.Length > 0)
        {
            agent.SetDestination(patrolPoints[currentPointIndex].position);
        }
        return;
    }

    if (distanceToPlayer < attackRange)
    {
        currentState = State.Attack;
        agent.isStopped = true;
        return;
    }
}

```

```

        agent.SetDestination(player.position);
    }

    private void Attack(float distanceToPlayer)
    {
        if (distanceToPlayer > attackRange)
        {
            currentState = State.Chase;
            agent.isStopped = false;
            return;
        }

        Vector3 direction = (player.position - transform.position).normalized;
        direction.y = 0;
        transform.rotation = Quaternion.LookRotation(direction);
    }

    private void GoToNextPoint()
    {
        if (patrolPoints.Length == 0) return;

        currentPointIndex = (currentPointIndex + 1) % patrolPoints.Length;
        agent.SetDestination(patrolPoints[currentPointIndex].position);
    }

    private void OnTriggerStay(Collider other)
    {
        if (other.CompareTag("Player") && currentState == State.Attack)
        {
            playerController.TakeDamage(attackDamage * Time.deltaTime);
        }
    }

    private void OnDrawGizmosSelected()
    {
        Gizmos.color = Color.yellow;
        Gizmos.DrawWireSphere(transform.position, detectionRange);

        Gizmos.color = Color.red;
        Gizmos.DrawWireSphere(transform.position, attackRange);
    }
}

```